

MODUAL-02 Collections, functions and Modules

[1] Accessing List

- 1) Understanding how to create and access elements in a list:
 - a) A list in Python is used to store multiple values in a single variable.
 - b) Features of List
 - i) Ordered
 - ii) Mutable (changeable)
 - iii) Indexed
 - iv) Allows duplicate values
 - c) Syntax
 - i) `list_name = [value1, value2, value3]`
 - d) Example
 - i) `fruits = ["apple", "banana", "mango"]`
 - e) Access Elements
 - i) `print(fruits[0])` # apple
 - ii) `print(fruits[-1])` # mango
 - f) Change Element
 - i) `fruits[1] = "orange"`
 - g) Add Element
 - i) `fruits.append("grapes")`
 - h) Remove Element
 - i) `fruits.remove("apple")`
 - i) Length of List
 - i) `print(len(fruits))`

2) Indexing in lists (positive and negative indexing)

a) Indexing in Lists (Positive and Negative Indexing)

i) What is Indexing?

Indexing is used to access elements of a list using their position number.

b) Positive Indexing

Positive indexing starts from 0 from the left side.

Example

```
fruits = ["apple", "banana", "mango"]
```

Index	Value
0	apple
1	banana
2	mango

Access Elements

```
print(fruits[0]) # apple
```

```
print(fruits[1]) # banana
```

```
print(fruits[2]) # mango
```

c) Negative Indexing

Negative indexing starts from -1 from the right side.

Index	Value
-1	mango
-2	banana
-3	apple

Access Elements

```
print(fruits[-1]) # mango
```

```
print(fruits[-2]) # banana
```

```
print(fruits[-3]) # apple
```

3) Slicing a list: accessing a range of elements.

i) Definition

(a) Slicing is used to access a range of elements from a list.

ii) Syntax

`list_name[start : stop : step]`

(a) start → Starting index

(b) stop → Ending index (not included)

(c) step → Interval between elements

iii) Example

```
numbers = [10, 20, 30, 40, 50, 60]
```

iv) Access Elements Using Slicing

(a) Range of Elements

```
print(numbers[1:4])
```

Output:

```
[20, 30, 40]
```

(b) From Beginning

```
print(numbers[:3])
```

Output:

```
[10, 20, 30]
```

(c) Up To End

```
print(numbers[2:])
```

Output:

```
[30, 40, 50, 60]
```

(d) Using Step

```
print(numbers[::2])
```

Output:

```
[10, 30, 50]
```

(e) Negative Slicing

```
print(numbers[-4:-1])
```

Output:

```
[30, 40, 50]
```

[2]List Operations

4) Common list operations: concatenation, repetition, membership.

(a) Common List Operations in Python

1. Concatenation (+)

Used to join two lists.

```
a = [1, 2]
```

```
b = [3, 4]
```

```
print(a + b)
```

Output:

```
[1, 2, 3, 4]
```

2. Repetition (*)

Used to repeat list elements.

```
a = [1, 2]
```

```
print(a * 3)
```

Output:

```
[1, 2, 1, 2, 1, 2]
```

3. Membership (in, not in)

Used to check whether an element exists in a list.

```
numbers = [10, 20, 30]
```

```
print(20 in numbers)
```

```
print(40 not in numbers)
```

Output:

```
True
```

```
True
```

5) Understanding list methods like append(), insert(), remove(), pop().

(a) List Methods in Python

1. append()

Used to add an element at the end of the list.

```
numbers = [1, 2, 3]
numbers.append(4)
```

```
print(numbers)
```

Output:

```
[1, 2, 3, 4]
```

2. insert()

Used to add an element at a specific position.

```
numbers = [1, 2, 4]
numbers.insert(2, 3)
```

```
print(numbers)
```

Output:

```
[1, 2, 3, 4]
```

3. remove()

Used to remove a specific element from the list.

```
numbers = [1, 2, 3]
numbers.remove(2)
```

```
print(numbers)
```

Output:

```
[1, 3]
```

4. pop()

Used to remove an element using index value.

```
numbers = [10, 20, 30]
numbers.pop(1)
```

[3] Working with Lists

6) Iterating over a list using loops

(a) Iterating Over a List Using Loops

1) Definition

Iterating means accessing each element of a list one by one using loops.

(b) Using for Loop

```
fruits = ["apple", "banana", "mango"]
```

```
for i in fruits:
```

```
    print(i)
```

Output:

apple

banana

mango

(c) Using range() and Index

```
fruits = ["apple", "banana", "mango"]
```

```
for i in range(len(fruits)):
```

```
    print(fruits[i])
```

Output:

apple

banana

mango

7) Sorting and reversing a list using sort(), sorted(), and reverse()

a) sort()

Used to sort the original list in ascending order.

```
numbers = [40, 10, 30, 20]
```

```
numbers.sort()
```

```
print(numbers)
```

Output:

```
[10, 20, 30, 40]
```

b) sorted()

Used to return a new sorted list without changing the original list.

```
numbers = [40, 10, 30, 20]
```

```
print(sorted(numbers))
```

Output:

```
[10, 20, 30, 40]
```

c) reverse()

Used to reverse the order of list elements.

```
numbers = [10, 20, 30, 40]
```

```
numbers.reverse()
```

```
print(numbers)
```

Output:

[40, 30, 20, 10]

8) Basic list manipulations: addition, deletion, updating, and slicing.

1. Addition

Used to add elements to a list.

```
numbers = [1, 2, 3]
```

```
numbers.append(4)
```

```
print(numbers)
```

Output:

```
[1, 2, 3, 4]
```

2. Deletion

Used to remove elements from a list.

```
numbers = [1, 2, 3]
```

```
numbers.remove(2)
```

```
print(numbers)
```

Output:

```
[1, 3]
```

3. Updating

Used to change list elements.

```
numbers = [10, 20, 30]
```

```
numbers[1] = 25
```

```
print(numbers)
```

Output:

```
[10, 25, 30]
```

4. Slicing

Used to access a range of elements.

```
numbers = [10, 20, 30, 40, 50] print(numbers[1:4])
```

[4] Tuple

9) Introduction to tuples, immutability.

1. Defining a Tuple

Tuples are written with **round brackets** () instead of the square brackets [] used for lists.

Python

A simple tuple

```
my_tuple = ("apple", "banana", "cherry")
```

A tuple can contain different data types

```
mixed_tuple = (1, "Hello", 3.14, True)
```

Note: To create a tuple with only one item, you must include a trailing comma (e.g., `single_item = (5,)`). Without the comma, Python will just see it as a number inside parentheses.

2. The Concept of Immutability

Immutability is the defining characteristic of a tuple. Once a tuple is created, you **cannot**:

- Change its values.
- Add new elements.
- Remove existing elements.

Why use an immutable object?

1. **Data Integrity:** It acts as a safeguard. If you have a set of constants (like the coordinates of a city or RGB color values), using a tuple ensures no part of your code accidentally overwrites them.

2. **Performance:** Because tuples are fixed in size, Python can allocate memory more efficiently. This makes them slightly faster to iterate through than lists.
3. **Dictionary Keys:** Because they are "hashable" (unchanging), tuples can be used as keys in dictionaries, whereas lists cannot.

3. Key Operations

Even though you can't change them, you can still interact with tuples in several ways:

- **Indexing:** Access items using their position: `my_tuple[0]` returns the first item.
- **Slicing:** Get a range of items: `my_tuple[1:3]`.
- **Unpacking:** Assign tuple elements to multiple variables at once.

Python

```
coordinates = (10, 20)
```

```
x, y = coordinates # x becomes 10, y becomes 20
```

4. When to Use Tuples vs. Lists

Feature	Tuple	List
Syntax	<code>(1, 2)</code>	<code>[1, 2]</code>
Mutability	Immutable (Fixed)	Mutable (Can change)
Performance	Faster	Slower
Use Case	Fixed data, records	Collections that grow or shrink

10) Creating and accessing elements in a tuple.

1. Creating Tuples

There are three primary ways to initialize a tuple in Python:

- Parentheses Method: The standard approach.

```
my_tuple = (10, 20, 30)
```

- Packing: You can create a tuple by simply separating values with commas without any brackets. Python "packs" them into a single object.

```
my_tuple = 10, 20, 30
```

- The tuple() Constructor: Converting other sequences (like lists or strings) into a tuple.

```
my_tuple = tuple([1, 2, 3])
```

2. Accessing Elements

Because tuples are ordered sequences, every item has a specific position called an index.

A. Positive Indexing

Counting starts at 0 from the left.

Python

```
colors = ("red", "green", "blue")  
print(colors[0]) # Output: "red"
```

B. Negative Indexing

Counting starts at -1 from the right. This is useful for accessing the last item without knowing the length of the tuple.

Python

```
print(colors[-1]) # Output: "blue"
```

C. Slicing

You can extract a "sub-tuple" by specifying a start and end range [start:stop]. Note that the stop index is exclusive (not included in the result).
Python

```
numbers = (0, 1, 2, 3, 4, 5)
print(numbers[1:4]) # Output: (1, 2, 3)
```

3. Practical Example: Unpacking

One of the most powerful ways to "access" tuple data is unpacking, where you extract all elements into individual variables in one line.
Python

```
# A tuple representing a point in 3D space
point = (4, 8, 12)
```

```
# Unpacking into variables
x, y, z = point
```

```
print(x) # 4
print(y) # 8
```

11) Basic operations with tuples: concatenation, repetition, membership.

1. Concatenation (+)

Concatenation allows you to join two or more tuples together to create a brand-new tuple.

- How it works: You use the + operator between two tuple objects.
- The Result: A third tuple containing all elements from the first, followed by all elements from the second.

Python

```
tuple1 = (1, 2, 3)
```

```
tuple2 = (4, 5)
```

```
combined = tuple1 + tuple2
```

```
# Output: (1, 2, 3, 4, 5)
```

2. Repetition (*)

Repetition allows you to repeat the contents of a tuple a specific number of times.

- How it works: You use the * operator followed by an integer.
- The Result: A new tuple where the original sequence is duplicated \$n\$ times.

Python

```
original = ("A", "B")
```

```
repeated = original * 3
```

```
# Output: ('A', 'B', 'A', 'B', 'A', 'B')
```

3. Membership (in and not in)

Membership operators allow you to check if a specific value exists within a tuple. This is a "Boolean" operation, meaning it returns either True or False.

- in: Returns True if the element is present.
 - not in: Returns True if the element is not present.
- Python

```
fruits = ("apple", "banana", "cherry")
```

```
print("apple" in fruits)    # Output: True  
print("orange" in fruits)  # Output: False  
print("grape" not in fruits) # Output: True
```

[5] Accessing Tuples

12) Accessing tuple elements using positive and negative indexing.

1. Positive Indexing (Left-to-Right)

Positive indexing is the standard way to access elements. It starts at 0 for the first item and increases by 1 for each subsequent item.

- First element: [0]
- Second element: [1]
- n -th element: [n-1]

Example:

Python

```
letters = ('A', 'B', 'C', 'D')  
# Index:  0  1  2  3
```

```
print(letters[0]) # Output: 'A'  
print(letters[2]) # Output: 'C'
```

2. Negative Indexing (Right-to-Left)

Negative indexing allows you to access elements starting from the end of the tuple. This is incredibly useful when you want to grab the last few items but don't want to calculate the total length of the tuple.

- Last element: [-1]
- Second-to-last: [-2]
- Third-to-last: [-3]

Example:

Python

```
letters = ('A', 'B', 'C', 'D')  
# Index: -4 -3 -2 -1
```

```
print(letters[-1]) # Output: 'D' (The last item)  
print(letters[-4]) # Output: 'A' (The first item)
```


3. Key Theoretical Concepts

- Zero-Based System: Always remember that positive indexing starts at 0, not 1. Forgetting this leads to "off-by-one" errors.
- IndexError: If you attempt to access an index that does not exist (e.g., trying to access index [5] in a tuple of length 3), Python will raise an IndexError: tuple index out of range.
- Immutability and Indexing: While you can read an element using an index, you cannot assign a new value to it.
 - `print(my_tuple[0])` is Allowed.
 - `my_tuple[0] = "New Value"` will trigger a TypeError.

13) Slicing a tuple to access ranges of elements.

1. The Three Components of a Slice

- start: The index where the slice begins (included). If omitted, it defaults to 0.
- stop: The index where the slice ends (excluded). If omitted, it goes to the very end of the tuple.
- step: Determines the increment between each index in the slice. If omitted, it defaults to 1.

2. Basic Slicing Examples

Consider the following tuple for all examples:

```
data = (10, 20, 30, 40, 50, 60)
```

A. Extracting a Range

To get elements from index 1 to 3:

Python

```
print(data[1:4])
```

Output: (20, 30, 40)

Note: Index 4 (50) is excluded.

B. Slicing from the Start or to the End

You can leave out the start or stop values to capture everything in that direction.

Python

```
print(data[:3]) # From the start to index 2: (10, 20, 30)
```

```
print(data[3:]) # From index 3 to the end: (40, 50, 60)
```

C. Using a Step

The step allows you to skip elements.

Python

```
print(data[0:6:2])
```

Output: (10, 30, 50) - every second element

3. Advanced Slicing Tricks

Negative Slicing

You can combine slicing with negative indexing to grab elements relative to the end of the tuple.

Python

Get the last three elements

```
print(data[-3:]) # Output: (40, 50, 60)
```

Reversing a Tuple

A common Python idiom for reversing a sequence is using a negative step of -1.

Python

```
print(data[::-1])
```

Output: (60, 50, 40, 30, 20, 10)

[6] Dictionaries

14) Introduction to dictionaries: key-value pairs.

1. The Key-Value Concept

Every entry in a dictionary consists of two parts:

- **Key:** A unique identifier used to find the data. It must be an immutable type (like a string, number, or tuple).
- **Value:** The data associated with that key. This can be anything—integers, strings, lists, or even other dictionaries.

The syntax uses curly braces `{ }` and colons `:` to separate keys from values.
Python

```
# A dictionary representing a user profile
user = {
    "username": "coder_123",
    "level": 5,
    "is_active": True
}
```

2. Core Characteristics

- **Uniqueness:** Keys must be unique within a single dictionary. If you assign a new value to an existing key, the old value is overwritten.
- **Mutability:** Unlike tuples, dictionaries are changeable. You can add, remove, or modify entries after the dictionary is created.
- **Mapping, not Sequence:** Unlike lists or tuples, dictionaries are not accessed via a numerical index (`$0`, `1`, `2...$`). Instead, they use a "mapping" system.

3. Basic Dictionary Operations

A. Accessing Values

You access a value by placing its key inside square brackets.

Python

```
print(user["username"]) # Output: coder_123
```

B. Adding or Updating

There is no special method required to add data; just assign a value to a key.

Python

```
user["level"] = 6      # Updates existing key  
user["email"] = "a@b.com" # Adds a new key-value pair
```

C. Removing Items

You can use the `del` keyword or the `.pop()` method.

Python

```
del user["is_active"]
```

4. Why use Dictionaries?

Dictionaries are incredibly efficient for looking up data. In technical terms, they use a Hash Table, which allows Python to find a value almost instantly, even if the dictionary contains millions of items. They are the go-to structure for:

- Database records: Representing a row of data.
- JSON data: The standard format for web APIs is structured exactly like Python dictionaries.
- Counting: Storing how many times different items appear in a list.

15) Accessing, adding, updating, and deleting dictionary element.

1. Accessing Elements

There are two primary ways to retrieve a value using its key:

- Square Brackets []: The most direct method.
 - *Risk*: If the key doesn't exist, Python raises a `KeyError`.
- The `.get()` Method: A safer way to access data.
 - *Benefit*: If the key is missing, it returns `None` (or a default value you specify) instead of crashing the program.

Python

```
user = {"name": "Alice", "age": 25}
```

```
print(user["name"])    # Output: Alice
print(user.get("email")) # Output: None (Safe)
```

2. Adding and Updating Elements

In Python, adding and updating use the exact same syntax. Python checks if the key already exists:

- If the key exists, the value is updated.
- If the key does not exist, a new pair is added.

Python

```
# Updating an existing key
user["age"] = 26
```

```
# Adding a new key-value pair
user["city"] = "New York"
```

3. Deleting Elements

There are several ways to remove data, depending on whether you need to use the removed value later.

- `del` keyword: Removes the key-value pair entirely.
- `.pop(key)`: Removes the key and returns the value. This is useful if you need to process the data one last time before it's gone.
- `.clear()`: Empties the entire dictionary, leaving it as `{}`.

Python

```
# Remove 'city' and ignore it
del user["city"]
```

```
# Remove 'age' and store the value
old_age = user.pop("age")
```

```
print(user) # Output: {'name': 'Alice'}
```

16) Dictionary methods like keys(), values(), and items().

1. keys()

The .keys() method returns a view object containing all the keys in the dictionary.

- Usage: Best used when you only care about the identifiers or labels.
- Example:

Python

```
car = {"brand": "Ford", "model": "Mustang", "year": 1964}
print(car.keys())
# Output: dict_keys(['brand', 'model', 'year'])
```

2. values()

The .values() method returns a view object containing all the values stored in the dictionary, ignoring the keys.

- Usage: Useful for calculations (like summing up prices) or checking if a specific piece of data exists regardless of its label.
- Example:

Python

```
print(car.values())
# Output: dict_values(['Ford', 'Mustang', 1964])
```

3. items()

The .items() method is perhaps the most useful. It returns a view of tuples, where each tuple is a (key, value) pair.

- Usage: This is the standard way to loop through a dictionary to get both the label and the data simultaneously.
- Example:

Python

```
print(car.items())
# Output: dict_items([('brand', 'Ford'), ('model', 'Mustang'), ('year', 1964)])
'''
```

[7] Working with Dictionaries

17) Iterating over a dictionary using loops.

1. Iterating Over Keys (The Default)

By default, when you loop over a dictionary directly, Python assumes you want the **keys**.

Python

```
inventory = {"apples": 50, "bananas": 30, "oranges": 20}
```

```
for item in inventory:
```

```
    print(item)
```

```
# Output: apples, bananas, oranges
```

Note: Using for item in inventory.keys(): does the exact same thing but is more explicit.

2. Iterating Over Values

If you only need the data and don't care about the labels, use the `.values()` method.

Python

```
for count in inventory.values():
```

```
    print(count)
```

```
# Output: 50, 30, 20
```

3. Iterating Over Key-Value Pairs (Recommended)

This is the most common "Pythonic" way to loop. Using the `.items()` method, you can **unpack** the key and the value into two separate variables in a single line.

Python

```
for item, count in inventory.items():
```

```
    print(f"We have {count} {item} in stock.")
```

```
# Output:
```

```
# We have 50 apples in stock.
```

```
# We have 30 bananas in stock.
```

```
# We have 20 oranges in stock.
```


4. Advanced: Modifying While Iterating

A common mistake is trying to add or remove items from a dictionary *while* looping over it. This will usually trigger a `RuntimeError`.

The Right Way: If you need to change the dictionary (like deleting items based on a condition), loop over a **copy** of the keys.

Python

18) Merging two lists into a dictionary using loops or zip().

1. Using the zip() Function (Recommended)

The zip() function takes two or more iterables and aggregates them into a single iterator of tuples. When passed into the dict() constructor, it creates a dictionary instantly.

- Syntax: dict(zip(keys, values))
- The Benefit: It is clean, readable, and highly optimized in Python.

Python

```
names = ["Alice", "Bob", "Charlie"]
scores = [85, 92, 78]
```

```
# Merging using zip
student_scores = dict(zip(names, scores))
```

```
print(student_scores)
# Output: {'Alice': 85, 'Bob': 92, 'Charlie': 78}
```

2. Using a for Loop

If you prefer a manual approach or need to add logic (like filtering) during the merge, you can use a loop. This usually requires the range() function to access the same index in both lists.

Python

```
names = ["Alice", "Bob", "Charlie"]
scores = [85, 92, 78]
```

```
student_scores = {}
```

```
for i in range(len(names)):
    student_scores[names[i]] = scores[i]
```

3. Using Dictionary Comprehension

A "Pythonic" middle ground between a loop and zip() is dictionary comprehension. This is useful if you want to modify the data as you merge it (e.g., giving everyone a 5-point bonus).

Python

```
# Merging with a bonus using comprehension
student_scores = {names[i]: scores[i] + 5 for i in range(len(names))}
```

19) Counting occurrences of characters in a string using dictionaries.

1. The Standard Loop Method

This is the most fundamental approach. You iterate through the string and check if the character is already in the dictionary.

- If it is: Increment the value by 1.
- If it isn't: Add the character to the dictionary with an initial value of 1.

Python

```
text = "hello world"
count_dict = {}
```

```
for char in text:
    if char in count_dict:
        count_dict[char] += 1
    else:
        count_dict[char] = 1
```

```
print(count_dict)
```

```
# Output: {'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}
```

2. The .get() Shortcut

You can simplify the if/else logic by using the .get() method. This method allows you to provide a default value (in this case, 0) if the key doesn't exist yet.

Python

```
count_dict = {}
for char in text:
    count_dict[char] = count_dict.get(char, 0) + 1
```

3. Using collections.Counter (The Professional Way)

If you are doing this in a real-world project, Python has a built-in tool called Counter from the collections module specifically for this purpose. It is highly optimized and much faster.

Python

```
from collections import Counter
```

```
text = "python programming"
```

[8] Functions

20) Defining functions in Python.

1. Basic Syntax

To define a function, you use the `def` keyword, followed by the function name, parentheses `()`, and a colon `:`. The code block inside must be indented.

Python

```
def greet_user():  
    """This is a docstring (optional) describing the function."""  
    print("Hello! Welcome to Python functions.")
```

```
# Calling the function  
greet_user()
```

2. Parameters and Arguments

Functions can accept data, which are called parameters when defined and arguments when passed.

- Parameters: The variables listed inside the parentheses in the function definition.
- Arguments: The actual values sent to the function when it is called.

Python

```
def personalize_greet(name):  
    print(f"Hello, {name}!")  
  
personalize_greet("Alice") # "Alice" is the argument
```

3. The return Statement

A function can send a value back to the caller using the `return` statement. Once a `return` is executed, the function exits immediately.

Python

```
def add_numbers(a, b):  
    return a + b  
  
result = add_numbers(5, 10)
```

```
print(result) # Output: 15
```

Note: If a function does not have a return statement, it implicitly returns None.

4. Default Parameter Values

You can provide default values for parameters. If the caller doesn't provide an argument for that parameter, the default is used.

Python

```
def describe_pet(pet_name, animal_type="dog"):
    print(f"I have a {animal_type} named {pet_name}.")

describe_pet("Willie")          # Uses default "dog"
describe_pet("Hammy", "hamster") # Overrides default
```

21) Different types of functions: with/without parameters, with/without return values.

1. No Parameters, No Return Value

These are often used for "utility" tasks, like printing a static message or clearing a screen. They do their job and don't need input or output.

Python

```
def say_hello():  
    print("Welcome to the system!")
```

```
say_hello()
```

```
# Output: Welcome to the system!
```

2. With Parameters, No Return Value

These functions are more flexible because they accept input data, but they still don't send a result back to the caller. They are common for logging data or updating a database.

Python

```
def print_square(number):  
    result = number * number  
    print(f"The square of {number} is {result}")
```

```
print_square(4)
```

```
# Output: The square of 4 is 16
```

3. No Parameters, With Return Value

These functions don't take input but generate or fetch a value to be used elsewhere. Think of a function that reads the current system time or generates a random number.

Python

```
import random
```

```
def get_random_die_roll():  
    return random.randint(1, 6)
```

```
roll = get_random_die_roll()
print(f"You rolled a {roll}!")
```

4. With Parameters, With Return Value (Most Common)

This is the standard "black box" function: you provide input, it performs a calculation or transformation, and it gives you a specific output.

Python

```
def calculate_tax(price, tax_rate):
    total = price * tax_rate
    return total
```

```
bill_tax = calculate_tax(100, 0.08)
print(bill_tax)
# Output: 8.0
```

22) Anonymous functions (lambda functions).

1. The Syntax

The structure of a lambda function is very specific:

lambda arguments: expression

- lambda: The keyword that initiates the function.
- arguments: The inputs (comma-separated if there are more than one).
- expression: A single piece of code that is executed and automatically returned.

Important: You cannot have multiple lines or complex statements (like if-else blocks or loops) inside a lambda—only a single expression.

2. Basic Example

Let's compare a regular function to a lambda function that squares a number:

Regular Function:

Python

```
def square(x):  
    return x * x
```

Lambda Function:

Python

```
square_lambda = lambda x: x * x
```

```
print(square_lambda(5)) # Output: 25
```

3. Why Use Lambda Functions?

The true power of lambdas isn't in assigning them to variables (like the example above), but in using them as throwaway functions inside other functions.

A. Sorting with a Custom Key

Suppose you have a list of tuples and want to sort them by the second element:

Python


```
pairs = [(1, 'one'), (3, 'three'), (2, 'two')]
pairs.sort(key=lambda x: x[1])
```

```
print(pairs) # Output: [(1, 'one'), (3, 'three'), (2, 'two')] (sorted alphabetically)
```

B. With filter()

Filtering a list to keep only even numbers:

Python

```
nums = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, nums))
```

```
# Output: [2, 4, 6]
```

C. With map()

Applying a transformation to every item in a list:

Python

```
doubled = list(map(lambda x: x * 2, nums))
```

```
# Output: [2, 4, 6, 8, 10, 12]
```

[9] Modules

23) Introduction to Python modules and importing modules.

1. What is a Module?

Think of a module like a toolbox. If you are building a house, you don't build a hammer from scratch; you go to your toolbox and take one out. In Python:

- A Library is a collection of toolboxes.
- A Module is a single toolbox (a .py file).
- A Function is a specific tool inside that toolbox.

2. How to Import Modules

There are several ways to bring a module into your current script using the import keyword.

A. The Basic Import

This imports the entire module. To use a function from it, you must use the "dot notation" (module.function).

Python

```
import math
```

```
print(math.sqrt(16)) # Output: 4.0
```

B. Importing Specific Attributes

If you only need one or two specific things from a module, you can use from. This allows you to use the function directly without the module prefix.

Python

```
from math import pi, sin
```

```
print(pi) # Output: 3.14159...
```

C. Using an Alias (as)

If a module name is long or you want to avoid naming conflicts, you can give it a nickname. This is very common in data science.

Python

24) Standard library modules: math, random.

1. The math Module

The math module provides access to mathematical functions defined by the C standard. It is used for precise calculations that go beyond basic arithmetic ($+$, $-$, $*$, $/$).

Common Constants

- `math.pi`: The mathematical constant π (approx. 3.14159).
- `math.e`: The mathematical constant e (approx. 2.71828).

Essential Functions

- Power & Roots: `math.sqrt(x)` returns the square root of x .
- Rounding:
 - `math.ceil(x)`: Rounds up to the nearest integer.
 - `math.floor(x)`: Rounds down to the nearest integer.
- Trigonometry: `math.sin(x)`, `math.cos(x)`, and `math.tan(x)` (expects x in radians).
- Factorial: `math.factorial(x)` returns $x!$.

Python

```
import math
```

```
print(math.sqrt(25)) # 5.0  
print(math.ceil(4.2)) # 5  
print(math.floor(4.8)) # 4
```

2. The random Module

While computers are deterministic, the random module implements pseudo-random number generators. This is essential for games, simulations, and testing.

Key Functions

- `random.random()`: Returns a random float between 0.0 and 1.0.
- `random.randint(a, b)`: Returns a random integer between a and b (inclusive of both).
- `random.choice(sequence)`: Picks one random element from a list, tuple, or string.

- `random.shuffle(list)`: Randomly reorders the elements of a list in-place.
- `random.uniform(a, b)`: Returns a random floating-point number between `a` and `b`.

Python

```
import random
```

```
# Generate a random integer for a dice roll  
print(random.randint(1, 6))
```

```
# Pick a random winner from a list  
players = ["Alice", "Bob", "Charlie"]  
print(random.choice(players))
```

25) Creating custom modules.

1. The 3-Step Process

Step 1: Create the Module

A module is just a regular .py file. Create a file named calculator.py and add some functions:

Python

```
# calculator.py
```

```
def add(a, b):  
    return a + b
```

```
def multiply(a, b):  
    return a * b
```

```
brand_name = "PyCalc 1.0"
```

Step 2: Save in the Same Directory

For the simplest setup, ensure your custom module (calculator.py) is in the same folder as the script that will use it.

Step 3: Import and Use

Create another file (e.g., main.py) and import your module using the filename (without the .py extension).

Python

```
# main.py
```

```
import calculator
```

```
print(calculator.add(10, 5))      # Output: 15
```

```
print(f"Using {calculator.brand_name}") # Output: Using PyCalc 1.0
```

2. The if __name__ == "__main__": Block

When you create a module, you might want to include some test code to make sure the functions work. However, you don't want that test code to run when someone else imports the module.

To prevent this, use the `__name__` check:
Python

```
# calculator.py
```

```
def add(a, b):  
    return a + b
```

```
if __name__ == "__main__":  
    # This only runs if you run calculator.py directly  
    # It will NOT run if you 'import calculator' elsewhere  
    print("Testing add function...")  
    print(add(10, 10))
```

3. Organizing Modules into Packages

As your project grows, you might have dozens of modules. You can group them into a folder, which Python calls a Package.

1. Create a folder named `my_tools`.
2. Place your `.py` files inside it.
3. (Optional in modern Python, but good practice) Add an empty file named `__init__.py` to tell Python the folder is a package.

To import from a package:
Python

```
from my_tools import calculator  
# OR  
import my_tools.calculator
```

4. Why Create Custom Modules?

- **Abstraction:** Your main script stays focused on high-level logic while the "dirty work" happens inside modules.
- **Encapsulation:** You can hide internal helper functions and only expose the ones you want others to use.
- **Collaboration:** Different team members can work on different modules simultaneously without conflicting in a single giant file.